

Deploy or Die — Produktrapport

Navn: Andrii Bondar · **Login:** anbo0005

Domæne: <https://andrii.mercantec.tech>

Repo: <https://github.com/Mercantec-GHC/deploy-or-die-anbo0005>

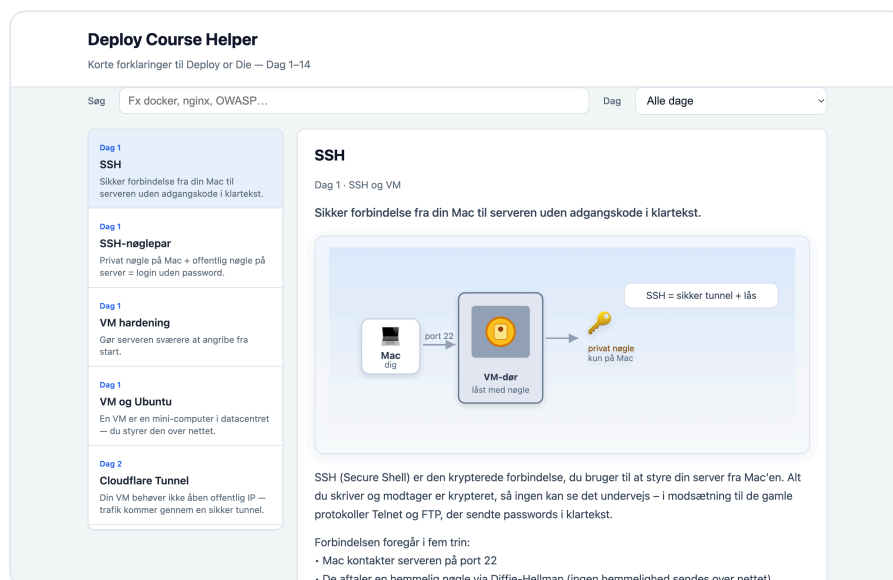
Kursus: Deploy or Die (14 dage) · Mercantec

1. Projektbeskrivelse

Hvad har jeg bygget/deployet?

Jeg har bygget og deployet **Deploy Course Helper** — en lille webapplikation, der fungerer som en interaktiv ordbog over kursets begreber (Dag 1–14). Den består af et ASP.NET Core 8 Web API med en statisk frontend (HTML/CSS/vanilla JS) serveret fra wwwroot/. Indholdet (kursusbegreber med forklaringer og illustrationer) ligger i `Data/terms.json` og hentes via API'et.

Endpoints: - `GET /api/terms` — alle begreber - `GET /api/terms/{id}` — ét begreb - `GET /api/terms/days` — dage/temaer - `GET /api/health` — health check (bruges af Uptime Kuma) - `GET /` — den statiske SPA



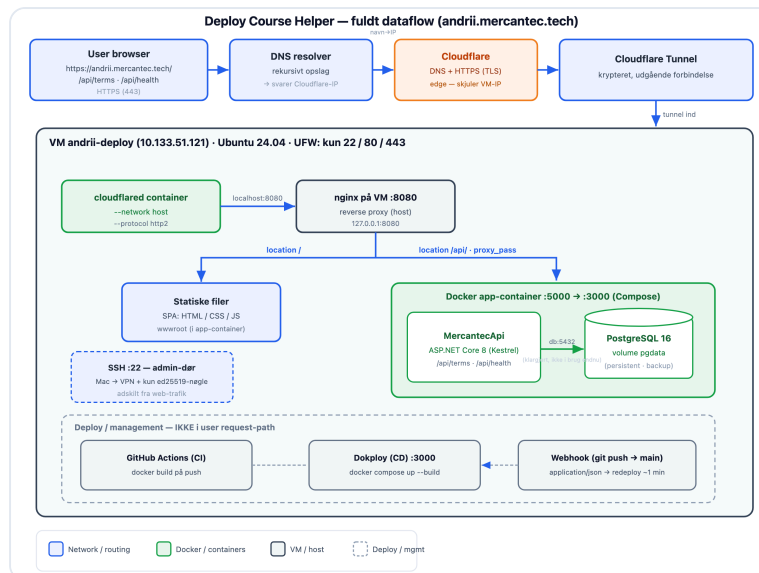
Skærbillede fra den kørende applikation (emnet SSH med metafor-illustration).

Vigtig afgrænsning: Mit fokus var bevidst lagt på **selve deploy-processen og infrastrukturen**, ikke på softwareudvikling. Applikationen er holdt simpel med vilje, så jeg fik mest muligt ud af deploy-materialet — pipeline, server, container, sikkerhed og drift — frem for at bruge tiden på en kompleks kodebase. Appen er reel og kørende, men den er midlet til at øve hele deploy-kæden, ikke målet i sig selv.

Tech stack

- **Backend:** ASP.NET Core 8 (Kestrel)
- **Frontend:** statisk HTML/CSS/JavaScript (ingen framework)
- **Database:** PostgreSQL 16 (`postgres:16-alpine`) — deployet i container med persistent volume.
Bemærk: appens kursusindhold læses pt. fra `Data/terms.json` ; databasen er sat op, verificeret (backup + persistence) og indgår som en del af deploy-infrastrukturen, men appen læser/skriver endnu ikke til Postgres
- **Container:** Docker Engine + Docker Compose v2
- **Webserver/proxy:** Nginx 1.24.0 på VM-hosten
- **Edge/HTTPS:** Cloudflare Tunnel (`cloudflared`)
- **CI:** GitHub Actions · **CD:** Dokploy
- **OS:** Ubuntu 24.04 LTS

2. Infrastruktur og Deployment



Figur: Det fulde dataflow fra brugerens browser hele vejen til app og database på VM'en.

Server

VM stillet til rådighed af Mercantec: Ubuntu 24.04 LTS, 4 CPU, 8 GB RAM, hostname `andrii-deploy`. Adgang via FortiClient VPN + SSH. Jeg **eksponerer ikke VM'en med en offentlig A-record eller åbne webporte** — al webtrafik kommer ind gennem Cloudflare Tunnel. (DHCP gav mig undervejs et IP-skift fra `.122` til `.121` efter en reboot, hvilket jeg måtte rette i min `~/ssh/config`.)

Domæne og HTTPS

Domænet `andrii.mercantec.tech` peger på Cloudflare (DNS styres af underviseren). Et `dig`-opslag returnerer Cloudflares IP'er, ikke VM'ens — det er korrekt for et tunnel-setup:

```
$ dig +short andrii.mercantec.tech
104.21.23.58
172.67.209.112
```

HTTPS termineres hos **Cloudflare**, så jeg behøvede ikke Certbot på VM'en. Trafikken flyder:

```
Bruger → HTTPS → Cloudflare → tunnel → cloudflared → nginx :8080 → app :5000
```

`cloudflared` kører som container med `--network host` og `--protocol http2` (QUIC blev blokeret på Mercantecs netværk).

Docker, Nginx og database

- **Nginx** (på hosten) lytter på `127.0.0.1:8080` og er reverse proxy: `location /` og `/api/` → `proxy_pass http://127.0.0.1:5000/`.
- **App + DB** kører via Docker Compose: app-containeren eksponeres på `127.0.0.1:5000` (Kestrel lytter på `:3000` inde i containeren), Postgres på `127.0.0.1:5432`.
- **Alt på localhost:** Hverken app-, db- eller adminporte er åbne mod internettet — kun nginx (via tunnel) er tilgængelig udefra.

Multi-stage Dockerfile: stage 1 (`sdk:8.0`) kompilerer med `dotnet publish`, stage 2 (`aspnet:8.0`) kører kun runtime. Det holder runtime-imaget lille (~320 MB), og .NET SDK behøver slet ikke at være installeret på VM-hosten.

`docker-compose.yml` definerer to services (`app`, `db`), et delt netværk (app når databasen via hostnavnet `db`, ikke `localhost`), named volume `pgdata`, `env_file: .env` og `depends_on: db (service_healthy)`.

```
# Compose-stack: app + database kører (db healthy)
$ docker compose ps
NAME                                STATUS
mercantecapi-...-app-1             Up
mercantecapi-...-db-1              Up (healthy)
```

3. CI/CD Pipeline

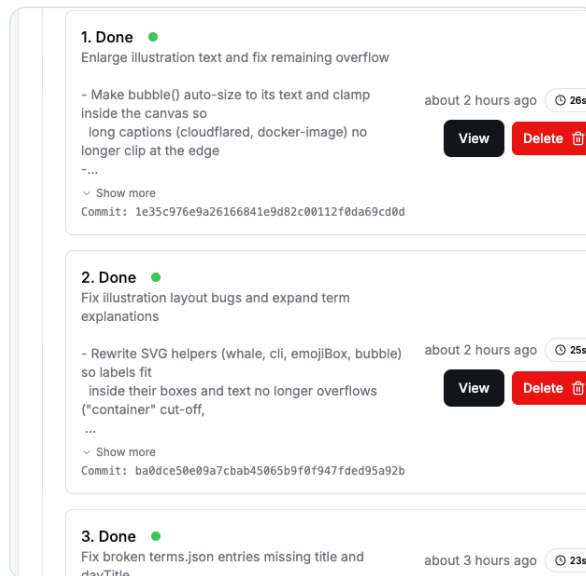
Hvordan deployer jeg?

Jeg har en automatiseret kæde fra min Mac til VM'en:

Mac (dotnet run lokalt) → git push main → GitHub Actions (CI) + Dokploy webhook (CD)

- **CI — GitHub Actions** (`.github/workflows/ci.yml`): kører ved `push` og `pull_request` mod `main`. Den bygger Docker-imaget (`docker build`) som kvalitetskontrol. Den rører ikke VM'en — den beviser kun, at imaget kan bygges.
- **CD — Dokploy** (selvhøstet på VM'en): koblet til repoet via GitHub PAT, branch `main`, med autodeploy. Et **webhook** fra GitHub kalder Dokploy ved hvert push, og Dokploy kører `docker compose up --build` på ~1 minut.

Begrænsning (ærligt): CI bygger pt. kun Docker-imaget — den kører endnu ikke unit tests, dependency-scan eller Trivy image-scan. Det er et naturligt næste skridt for en mere moden pipeline.



Dokploy Deployments: hvert push til `main` udløser et automatisk redeploy — her med commit-hash, status og byggetid.

Versionsstyring og projektstyring

- Ét Git-repo til alt: `app/` (kode + Dockerfile + compose) og `docs/` (noter, workflow, log, handoff). Branch-strategi: `main` (solo-projekt, lille scope).
- Jeg fører en **pre-commit checkliste** i `docs/WORKFLOW.md` (ingen secrets, ingen `bin/obj`, tjek `git status` / `diff` før commit).
- Projektstatus og fremdrift holdes i `docs/DOCS_INDEX.md` (✅/❌ pr. dag), `docs/SESSION_HANDOFF.md` (genoptag-fil mellem sessioner) og `docs/DEPLOY_RESULTS_LOG.md` (resultater pr. dag). Jeg har ikke brugt GitHub Projects — dokumentationen i repoet er min tracking.

4. Sikkerhed

Jeg har gennemført flere konkrete tiltag (verificeret), og enkelte punkter forblev teori.

Implementeret: - **SSH-hardening:** kun ed25519-nøgle, `PermitRootLogin no`, `PasswordAuthentication no` i `sshd_config`. Verificeret med `Accepted publickey` i `sshd-loggen`. - **UFW firewall:** `deny incoming` / `allow outgoing`, kun **22, 80, 443** åbne. - **Cloudflare Tunnel:** ingen offentlig IP/åbne webporte på VM'en — angrebsfladen er minimal. Postgres og adminporte (Dokploy, Uptime Kuma) lytter kun på `127.0.0.1`. - **Secrets:** DB-credentials i `.env` (i `.gitignore`, aldrig i git). - **Security headers** (Dag 11, nginx): CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy — bekræftet med `curl -I`. - **OWASP-perspektiv på eget projekt:** A02 (secrets ude af git, TLS via Cloudflare), A05 (headers + localhost-kun database). `/api/` har ingen auth, hvilket er bevidst OK for en offentlig læse-demo.

```
# Firewall – kun de nødvendige porte er åbne
$ sudo ufw status
Status: active
22/tcp      ALLOW      Anywhere
80/tcp      ALLOW      Anywhere
443/tcp     ALLOW      Anywhere
```

```
# Security headers leveret af nginx (Dag 11)
$ curl -sI https://andrii.mercantec.tech/ | head
HTTP/2 200
content-security-policy: default-src 'self' ...
strict-transport-security: max-age=...
x-frame-options: DENY
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
```

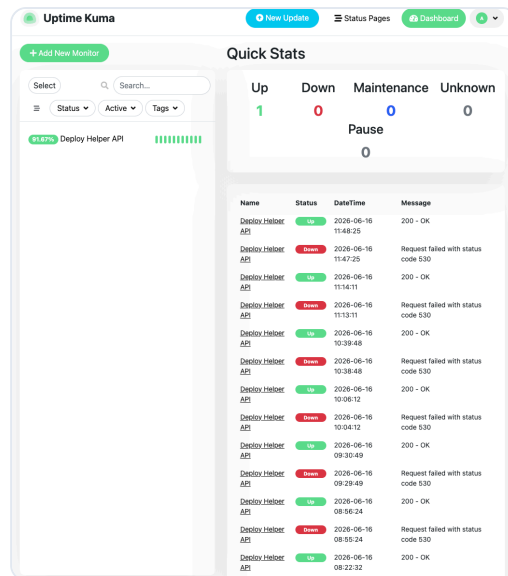
Sikkerhedshændelse (åben opgave): En GitHub PAT blev utilsigtet synlig på et screenshot under arbejdet — en reel risiko (eksponeret credential). **Status: ikke afsluttet endnu** — token skal revokes/roteres på GitHub. Læringen er, at det ikke er nok at fjerne en token fra en note, fordi historik og screenshots stadig findes; mitigeringen er først færdig, når selve token er gjort ugyldig.

Teori / ikke kørt på dette projekt: Trivy image-scanning, `dotnet list package --vulnerable`, Certbot (unødvendigt ved tunnel), NIS2/CRA (mundtligt Dag 2).

5. Monitoring og Drift

Hvordan holder jeg øje med løsningen?

- **Dokploy UI** (`https://andriidokploy.mercantec.tech`): container-status (Running/Exited), ressourcer og deploy-historik — set *indefra* VM'en.
- **Uptime Kuma** (`louislam/uptime-kuma`, port `127.0.0.1:3001`, tilgået via SSH-port-forward): monitorerer `GET https://andrii.mercantec.tech/api/health` hvert 60. sekund. Status **Up · 200 OK**. Kuma tester hele vejen udefra (Cloudflare → tunnel → nginx → app) — det Dokploy ikke kan se.
- **App-niveau:** `/api/health` svarer 200 JSON (uptime-signal), et ukendt begreb giver 404, og runtime-fejl ses i ASP.NET stdout via Dokploy Logs.
- **Logs:** ASP.NET stdout via `docker compose logs` og Dokploy Logs.
- **Backup:** `pg_dump` → `backup_20260615.sql`; persistens verificeret med `docker restart db` + `SELECT 1`.



Uptime Kuma: monitoren rammer `/api/health` hvert 60. sekund. De periodiske "530"-hændelser er netop Cloudflare-tunnelen, der falder (se afsnit 6) — monitoreringen fanger det udefra, som en bruger ville opleve det.

Begrænsning: Kuma kører selv som container på *samme* VM. Den tester ganske vist den fulde eksterne sti (ud via Cloudflare-tunnelen og tilbage), men den er ikke en helt uafhængig observatør: går hele VM'en eller dens netværk ned, er Kuma også nede og kan ikke alarmere. En ekstern third-party-uptime-tjeneste på et andet netværk ville være mere robust — et naturligt næste skridt.

Fejlhåndtering i praksis

Jeg lærte at læse statuskoder som diagnose: - **502** = tunnel OK, men nginx/app svarer ikke (fx app-container ikke startet endnu). - **530** / "**Lost connection with edge**" = tunnel tabte forbindelsen → fix: `docker restart cloudflared`. - **1033** = `cloudflared` kører ikke.

6. Læring og Refleksion

Hvad gik godt

- Hele kæden kom til at hænge sammen og blev verificeret trin for trin: SSH/UFW (Dag 1–2) → Docker + Postgres + tunnel (Dag 3) → nginx reverse proxy (Dag 4–5) → app i container (Dag 6) → compose-stack (Dag 7) → CI + CD (Dag 8) → backup, monitoring og headers (Dag 9–11).
- Jeg testede altid i rækkefølge `:5000` → `:8080` → `https://domæne`, så jeg vidste præcis, *hvilket lag* en fejl lå i:

```
$ curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:5000/api/terms # app → 200
$ curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080/api/terms # nginx → 200
$ curl -s -o /dev/null -w "%{http_code}\n" https://andrii.mercantec.tech/api/terms # tunnel → 200
```

- CI/CD-flowet virkede: push til `main` → grøn CI → Dokploy redeploy på ~1 min.

Hvad gik skævt

- **502 ved tunnel-start:** `cloudflared` i bridge-netværk pegede på containerens eget localhost. Fix: `--network host`.
- **QUIC blokeret:** måtte tvinge `--protocol http2`.
- **Cloudflare Tunnel faldt jævnlgt — min mest vedvarende driftsudfordring.** Med jævne mellemrum mistede `cloudflared` sin edge-forbindelse (`Lost connection with edge`), så domænet svarede **530**, selvom `curl http://127.0.0.1:8080` lokalt gav 200. Det viste, at fejlen lå i tunnel-laget — ikke i app eller nginx. Workaround var `docker restart cloudflared`, hvorefter loggen igen viste `Registered tunnel`

`connection` . En mere robust løsning ville være at køre cloudflared som auto-genstartende service med health-overvågning, så den selv kommer op igen uden manuel indgriben.

- **Webhook-encoding:** GitHub default `form` gav "Branch Not Match" i Dokploy. Fix: `Content-Type application/json` .
- **GitHub Actions scope:** første push af workflow fejlede pga. manglende `workflow` -scope på token. Fix: `gh auth refresh -s workflow` .
- **Forkert server-fingerprint — samme IP pegede på to forskellige VM'er.** Den samme interne IP (`10.133.51.122`) svarede på forskellige forsøg med to forskellige ED25519-fingerprints. Den forkerte host (`SHA256:P4Z...`) lod mig svare på prompten, men afviste derefter med `Permission denied (publickey)` — det var en anden elevs VM, som DHCP havde tildelt samme IP, og den havde ikke min nøgle. Den rigtige VM (`SHA256:MFyp...`) loggede mig direkte ind med `Welcome to Ubuntu 24.04.4 LTS` . Fingerprintet var altså den eneste måde at skelne de to maskiner på. (Senere skiftede mit IP desuden fra `.122` til `.121` efter en reboot.) Læring: verificér altid fingerprintet — accepter ikke bare blindt "yes" — og ryd den forældede host key med `ssh-keygen -R` :

```
$ ssh-keygen -R 10.133.51.122          # ryd forældet host key

# Forsøg 1 — forkert VM (samme IP, anden maskine):
$ ssh mercantec-andrii
ED25519 key fingerprint is SHA256:P4Z/CQ6Zhu1V+QL+ZlhGh487DWYnWCFKo2ssu05Bs2k
Are you sure you want to continue connecting? yes
andrii@10.133.51.122: Permission denied (publickey).

# Forsøg 2 — den rigtige VM:
$ ssh mercantec-andrii
ED25519 key fingerprint is SHA256:MFypL9BQUdg1oJq/HkYZxtcv7/w8wvXB0ZwRoVjD0Ho
Are you sure you want to continue connecting? yes
Welcome to Ubuntu 24.04.4 LTS
```

Hvad har jeg lært

Teknisk forstår jeg nu deploy-kæden som **adskilte lag**, der hver kan fejle og fejlfindes for sig: SSH (:22, kun nøgle) er en anden "dør" end webtrafikken (tunnel → nginx → app). Dockerfile (*hvordan bygges imaget*) og compose (*hvad køres sammen*) er ikke dubletter. `localhost` inde i en container er ikke VM-hosten. Og CI (bygger koden) er ikke det samme som CD (deployer til VM).

Refleksion over scope: ved bevidst at holde applikationen enkel kunne jeg gennemføre og faktisk *verificere* hvert deploy-trin — i stedet for at drukne i app-kode. Det gav mig mest mulig læring ud af deploy-materialet.

Dette var et **solo-projekt**, så samarbejdsdelen handlede primært om disciplin med mig selv: at dokumentere hver session (handoff-fil), føre en resultatlog og holde en fast commit-/verifikationsrutine, så jeg kunne genoptage arbejdet uden at tabe tråden.

Næste skridt: koble appen reelt på Postgres (læse/skrive begreber fra databasen), tilføje unit tests og dependency-/image-scanning (Trivy) i CI, og afslutte PAT-rotationen.